MACHINE LEARNING EXAM PAPER 2024

PLEASE ANSWER 3 OUT OF 4 QUESTIONS.

1. Linear Classification, Closed form solution, Non-linear Feature Transformation, VC Dimension & Perceptron Learning Algorithm

a) You are given a dataset with 600 samples of movies, each rated on a scale from 1 to 5 (see Table 1 for sample data entries). The dataset contains the following features for each movie:

- Movie Age: Age of the movie in years (calculated from January 2024).
- Movie Length: Duration of the movie in hours.
- Movie Budget: Movie budget in millions of pounds.
- Box Office: Box office earnings in millions of pounds.

Movie Age	Movie Length	Movie Budget	Box Office	Rating
5	2	10	25	4
2	1.5	7.5	15	3
15	2.5	5	10	2
1	1.8	15	30	5
⋮	⋮	⋮	⋮	⋮
2	1.5	7.5	15	2

Table 1 Sample of movie rating data.

Your task is to predict the movie rating using historical data including features and labels. Answer the following:

i) Is this a supervised or unsupervised machine learning problem? Explain why. **[2 marks]**

Solution: It is supervised learning problem since we are using both features and labels in prediction.

ii) How can we tackle this problem either as regression or classification? Justify your answer. **[2 marks]**

Solution: The rating can be considered as discrete values 1,2,3,4,5 in this case we can consider it a classification problem. The rating can

be considered also a continuous value between 1 and 5 which can be considered a regression problem.

iii) Explain when can stochastic noise occur. Give an example from your dataset. **[2 marks]**

Solution: Stochastic noise occurs when the target function cannot be modeled as a function but as a probability of y given x. We can see this clear in the sample $[2, 1.5, 7.5, 15]$ since that it was labeled as both 3 and 2 and therefore cannot be modeled as a function.

iv) Suppose you decide to simplify the prediction by classifying movies as "positive" if their rating is 3 or higher and "negative" if their rating is below 3. Assume you are using a Perceptron Learning Algorithm (PLA) with the same feature space to solve this binary classification problem, answer the following:

What will be the VC dimension of the hypothesis class when applying the perceptron model given? **[3 marks]**

Solution: The VC dimension in this case will be $d + 1 = 5$ where d is the feature space dimension.

Using the rule of thumb learned in the class, comment whether the data samples are considered sufficient to train the perceptron. **[3 marks]**

Solution: The rule of thumb says $n \geq 10 \cdot d_{vc}$. The VC dimension is 5 and we have 600 samples which satisfy $600 \geq 500$

Describe the main challenge you may encounter in finding the best hypothesis using the PLA and suggest two solutions to reduce it (assuming that you have to use PLA). **[3 marks]**

Solution: The main challenge would be for the data not being linearly separable, one solution is to use pocket algorithm to pick the hypothesis that minimizes the error. Another solution would be to use non-linear transformation.

b) Consider the truth table of AND gate given in Table 2.

$\mathbf{x_1}$	$\mathbf{x_2}$	$\mathbf{y}$
-1	-1	-1
-1	+1	-1
+1	-1	-1
+1	+1	+1

Table 2 Truth table for AND gate.

Suppose you want to use a perceptron to build an AND gate classifier. Apply the perceptron learning algorithm (PLA) you learned in the class to find the parameters of this perceptron. Then prove why the resulting hypothesis by PLA and the perceptron given by $h \rightarrow [w_0, w_1, w_2] = [-1.5, 1, 1]$ can be both accepted as a solution. **[15 marks]**

Solution:

$\mathbf{x}_1 = (1,-1,-1), \mathbf{x}_2 = (1,-1,1), \mathbf{x}_3 = (1,1,-1), \mathbf{x}_4 = (1,1,1)$
$\dot{\mathbf{w}} = (w_0, w_1, w_2) = ?$ for setting $\mathbf{x}$ and $\dot{\mathbf{w}}$
find $\text{sign}(\mathbf{w}^\top \mathbf{x}_i) \neq y_i$, update $\mathbf{w} \leftarrow \mathbf{w} + y_i \mathbf{x}_i$ for formulas
$\mathbf{w}_{(1)} = (w_0, w_1, w_2) = (0,0,0)$, $\mathbf{w}_{(1)}^T \mathbf{x}_1 = 0 \Rightarrow +1 \neq y_1$
$\mathbf{w}_{(2)} = \mathbf{w}_{(1)} + y_1 \mathbf{x}_1 = (-1,1,1)$
$\mathbf{w}_{(2)}^T \mathbf{x}_1 = -3 \Rightarrow -1 = y_1$
$\mathbf{w}_{(2)}^T \mathbf{x}_2 = -1 \Rightarrow -1 = y_2$
$\mathbf{w}_{(2)}^T \mathbf{x}_3 = -1 \Rightarrow -1 = y_3$
$\mathbf{w}_{(2)}^T \mathbf{x}_1 = 1 \Rightarrow +1 = y_4$
$\dot{\mathbf{w}} = \mathbf{w}_{(2)} = (-1,1,1)$

In perceptron, any hypothesis that can seperate the two classes is considered a solution. Both $h \rightarrow [-1.5, 1, 1]$ and $h \rightarrow [-1, 1, 1]$ belong to the same dichotomy in this dataset.

c) Consider a classifier called positive-ray that is given by the following hypothesis:

$$h(x) = \text{sign}(x - a) \text{ s.t. } x \in \mathbb{R}^1$$

Point a acts as decision boundary in 1-D space, where any data point can be only classified within the positive class if it lies exactly on a or on its right hand side (see Figure 1.1).

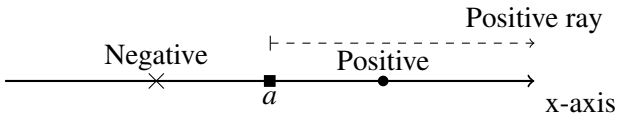


Figure 1.1 Positive-ray classifier.

Using the bound $m_{\mathscr{H}} = \sum_{i=0}^{k-1} \binom{N}{i}$, where k is the break-point, derive the polynomial that represents the maximum number of dichotomies that can be obtained using this classifier in terms of N (the number of samples). Then using the polynomial obtained find the maximum number of dichotomies when $N = 100$. **[15 marks]**

Solution: By the classifier definition, this classifier cannot get all the possible dichotomies on 2 data points and therefore The break-point is $k = 2$. The maximum number of dichotomies can be found by applying the growth function formula:

$$m_{\mathscr{H}} = \sum_{i=1}^{k-1} \binom{N}{i}$$

By substituting $k = 2$ in the above equation we get:

$$\begin{aligned} m_{\mathscr{H}} &= \sum_{i=1}^{1} \binom{N}{i} \\ &= \binom{N}{0} + \binom{N}{1} \\ &= \frac{N!}{N! \cdot 0!} + \frac{N!}{(N-1)! \cdot 1!} \\ &= N + 1 \end{aligned}$$

By applying $N = 100$ in the above equation we get the maximum number of dichotomies:

$$\begin{aligned} m_{\mathscr{H}} &= 100 + 1 \\ &= 101 \end{aligned}$$

d) Consider the data samples along with their ground truth values y shown in Table 3.

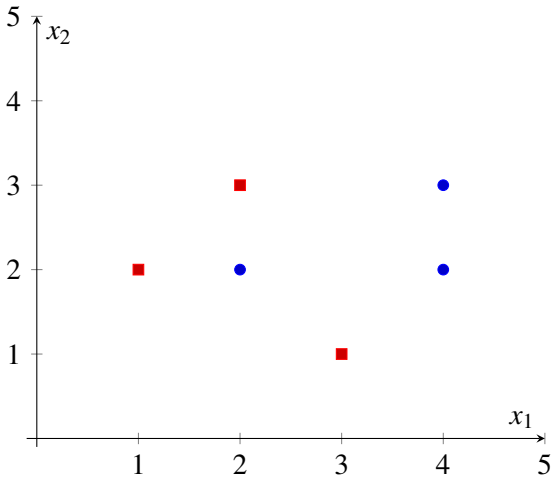
i) Plot the data on a Cartesian plane and prove that they are non-linearly separable and cannot be classified with zero error using a linear classifier (e.g., a perceptron). **[10 marks]**

x_1	x_2	y
2	2	-1
4	2	-1
4	3	-1
1	2	+1
3	1	+1
2	3	+1

Table 3 Data sample.

Solution:

By plotting the data sample points on Cartesian plane, we can easily observe that the two classes cannot be separated with a line.



ii) Apply the regression for classification approach by using closed form solution to find the weight vector. State whether you would consider the hypothesis you found as good solution by estimating the correctly classified and misclassified samples and commenting on your findings. **[20 marks]**

Solution:

$$\mathbf{W} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}$$

$$
= \left(\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 2 & 4 & 4 & 1 & 3 & 2 \\ 2 & 2 & 3 & 2 & 1 & 3 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 2 \\ 1 & 4 & 2 \\ 1 & 4 & 3 \\ 1 & 1 & 2 \\ 1 & 3 & 1 \\ 1 & 2 & 3 \end{bmatrix} \right)^{-1} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 2 & 4 & 4 & 1 & 3 & 2 \\ 2 & 2 & 3 & 2 & 1 & 3 \end{bmatrix} \cdot \begin{bmatrix} -1 \\ -1 \\ -1 \\ +1 \\ +1 \\ +1 \end{bmatrix}
$$

$$
= \frac{1}{124} \cdot \begin{bmatrix} 325 & -41 & -90 \\ -41 & 17 & -2 \\ -90 & -2 & 44 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 2 & 4 & 4 & 1 & 3 & 2 \\ 2 & 2 & 3 & 2 & 1 & 3 \end{bmatrix} \cdot \begin{bmatrix} -1 \\ -1 \\ -1 \\ +1 \\ +1 \\ +1 \end{bmatrix}
$$

$$
= \frac{1}{124} \cdot \begin{bmatrix} 63 & -19 & -109 & 104 & 112 & -27 \\ -11 & 23 & 21 & -28 & 8 & -13 \\ -6 & -10 & 34 & -4 & -52 & 38 \end{bmatrix} \cdot \begin{bmatrix} -1 \\ -1 \\ -1 \\ +1 \\ +1 \\ +1 \end{bmatrix}
$$

$$
= \frac{1}{124} \cdot \begin{bmatrix} 254 \\ -66 \\ -36 \end{bmatrix}
$$

$$
\approx \begin{bmatrix} 2.05 \\ -0.53 \\ -0.29 \end{bmatrix}
$$

Applying the hypothesis on the training data to get correctly classified and misclassified samples:

$$
\begin{aligned}
\text{Point}[2,2] &\rightarrow \text{sign}\,[2.05 + -0.53 \cdot 2 + -0.29 \cdot 2] \\
&= \text{sign}[0.41] = +1 \neq y_1 \text{ Misclassified} \\
\text{Point}[4,2] &\rightarrow \text{sign}\,[2.05 + -0.53 \cdot 4 + -0.29 \cdot 2] \\
&= \text{sign}[-0.65] = -1 = y_2 \text{ Correctly classified} \\
\text{Point}[4,3] &\rightarrow \text{sign}\,[2.05 + -0.53 \cdot 4 + -0.29 \cdot 3] \\
&= \text{sign}[-0.94] = -1 = y_3 \text{ Correctly classified} \\
\text{Point}[1,2] &\rightarrow \text{sign}\,[2.05 + -0.53 \cdot 1 + -0.29 \cdot 2] \\
&= \text{sign}[0.94] = +1 = y_4 \text{ Correctly classified}
\end{aligned}
$$

$$\begin{aligned}
\text{Point}[3,1] &\rightarrow \text{sign}\,[2.05 + -0.53 \cdot 3 + -0.29 \cdot 1] \\
&= \text{sign}[0.17] = +1 = y_5 \text{ Correctly classified} \\
\text{Point}[2,3] &\rightarrow \text{sign}\,[2.05 + -0.53 \cdot 2 + -0.29 \cdot 3] \\
&= \text{sign}[0.12] = +1 = y_6 \text{ Correctly classified}
\end{aligned}$$

Despite of the data being non-linearly separable; however, this approach has only one misclassified point which is the best we can get when we have non-linearly separable data.

iii) Apply the non-linear transformations below:

$$\Phi(\mathbf{x}) = \begin{cases} \left(x_1 x_2,\ \frac{x_1^2}{4}\right) & \text{if } (x_1 x_2 \bmod 4) = 0 \\ (x_1,\ x_2) & \text{if } (x_1 x_2 \bmod 4) \neq 0 \end{cases}$$

Note: mod is the operation that returns the remainder of the division.

and answer the following:

- State whether this transformation can make the data linearly separable. Then mention if this transformation retained the original feature space or not. **[15 marks]**

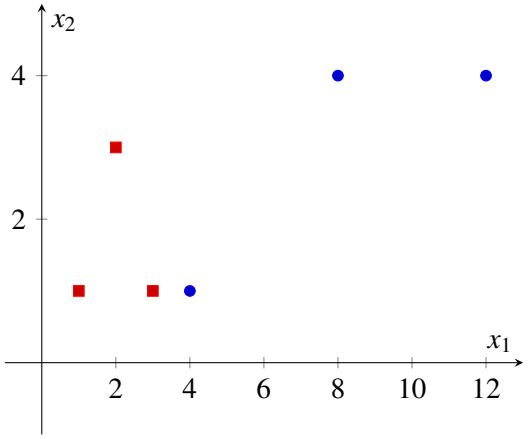
Solution: For this transformation Φ, we can notice that for all the negative class examples, the condition $(x_1 x_2 \bmod 4) = 0$ holds. Therefore, we can estimate the transformed points by applying $\left(x_1 x_2,\ \frac{x_1^2}{4}\right)$ as follows:

$$\begin{aligned}
\Phi\begin{pmatrix}2\\2\end{pmatrix} &= \begin{pmatrix}2\cdot 2\\ \frac{2^2}{4}\end{pmatrix} = \begin{pmatrix}4\\1\end{pmatrix} \\
\Phi\begin{pmatrix}4\\2\end{pmatrix} &= \begin{pmatrix}4\cdot 2\\ \frac{4^2}{4}\end{pmatrix} = \begin{pmatrix}8\\4\end{pmatrix} \\
\Phi\begin{pmatrix}4\\3\end{pmatrix} &= \begin{pmatrix}4\cdot 3\\ \frac{4^2}{4}\end{pmatrix} = \begin{pmatrix}12\\4\end{pmatrix}
\end{aligned}$$

On the other hand, for all the positive class examples, the condition $(x_1 x_2 \bmod 4) \neq 0$ holds. Therefore, we can estimate the transformed points by applying $(x_1,\ x_2)$ as follows:

$$\Phi\begin{pmatrix}1\\2\end{pmatrix} = \begin{pmatrix}1\\2\end{pmatrix}$$
$$\Phi\begin{pmatrix}3\\1\end{pmatrix} = \begin{pmatrix}3\\1\end{pmatrix}$$
$$\Phi\begin{pmatrix}2\\3\end{pmatrix} = \begin{pmatrix}2\\3\end{pmatrix}$$

Plotting the transformed points on the Cartesian space to get the following:

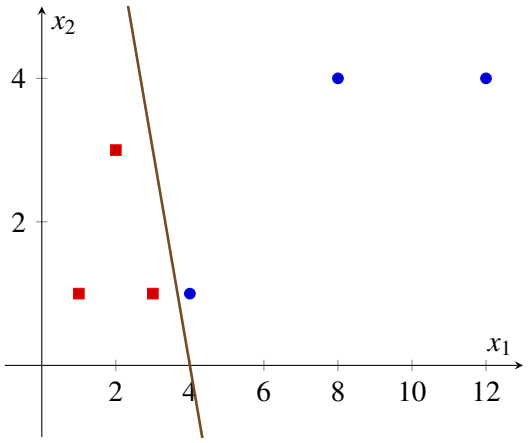


From the plot we can observe that the data became linearly separable after applying the transformation. And the transfomration retained the original feature space (both of them are 2D).

- Suppose that there is a perceptron to classify the newly transformed points given by its weights $[w_0, w_1, w_2, ..., w_n]$ as follows $h \rightarrow \mathbf{w} = [12, -3, -1]$. Estimate the confusion matrix by applying the hypothesis (reporting true positives, true negatives, false positives and false negatives) and total accuracy. **[10 marks]**

Solution:

We can plot the data points against the decision boundary as follows in order to specify TP, FP, TN, FN:



According to the plot all the points are classified correctly such that $TP = TN = 3$ and $FP = FN = 0$.

		Predicted	
		Positive	Negative
Actual	Positive	3	0
	Negative	0	3

$$\begin{aligned} Accuracy &= \frac{TP+TN}{TP+TN+FP+FN} \\ &= \frac{3+3}{3+3+0+0} \\ &= 1 \end{aligned}$$

2. Linear regression, Neural Networks, Backpropagation, Gradient Descent, Regularization, Validation, and Bias-Variance Analysis

a) Consider a sample dataset of Iris flowers with features "sepal length" and "petal length", and target classes "Setosa" (labeled as 1) and "Virginica" (labeled as 0). Table 4 contains the training samples extracted from this dataset.

Sepal Length in cm ($\mathbf{x_1}$)	Petal Length in cm ($\mathbf{x_2}$)	Flower Type ($\mathbf{y}$)
5	1.5	1
6	5	0

Table 4 Training samples.

i) Train a logistic regression classifier to classify the training data using batch gradient descent over two steps with a learning rate of 0.1. Assume all the weights are initialised with zero at the beginning and the binary cross entropy function that you will use is given by:

$$\text{BCE} = \frac{1}{N}\sum_{i=1}^{N}[-y_i \log(\hat{y}_i) - (1-y_i)\log(1-\hat{y}_i)]$$

In your solution, state clearly the gradient, update rules used and the updated weights after each step. **[20 marks]**

Solution:

The update rule in gradient descent is given by:

$$W_{new} = W_{old} - \alpha \cdot \frac{\partial BCE}{\partial W}$$

The gradient is given by the following, considering BCE, for each weight such that j refer to the parameter index:

$$\begin{aligned}\frac{\partial BCE}{\partial w_j} &= \frac{1}{N}\sum_{i=1}^{N}\left[\frac{\partial BCE}{\partial \hat{y}_i}\cdot\frac{\partial \hat{y}_i}{\partial z_i}\cdot\frac{\partial z_i}{\partial w_j}\right] \\ &= \frac{1}{N}\sum_{i=1}^{N}[(\hat{y}_i - y_i)\cdot x_{i,j}]\end{aligned}$$

Calculating predictions on intialization: For $x_1 = (5, 1.5)$:

$$z_1 = w_0 + w_1 \cdot 5 + w_2 \cdot 1.5 = 0 + 0 \cdot 5 + 0 \cdot 1.5 = 0$$

$$\hat{y}_1 = \sigma(z_1) = \frac{1}{1+e^{-0}} = 0.5$$

For $x_2 = (6,5)$:

$$z_2 = w_0 + w_1 \cdot 6 + w_2 \cdot 5 = 0 + 0 \cdot 6 + 0 \cdot 5 = 0$$
$$\hat{y}_2 = \sigma(z_2) = \frac{1}{1+e^{-0}} = 0.5$$

Step 1:

Gradient with respect to w_0 (bias term):

$$\frac{\partial L}{\partial w_0} = \frac{1}{2}\left((0.5-1)\cdot 1 + (0.5-0)\cdot 1\right)$$
$$= \frac{1}{2}(-0.5+0.5) = 0$$

Gradient with respect to w_1:

$$\frac{\partial L}{\partial w_1} = \frac{1}{2}\left((0.5-1)\cdot 5 + (0.5-0)\cdot 6\right)$$
$$= \frac{1}{2}(-0.5\cdot 5 + 0.5\cdot 6) = \frac{1}{2}(-2.5+3) = \frac{1}{2}\cdot 0.5 = 0.25$$

Gradient with respect to w_2:

$$\frac{\partial L}{\partial w_2} = \frac{1}{2}\left((0.5-1)\cdot 1.5 + (0.5-0)\cdot 5\right)$$
$$= \frac{1}{2}(-0.5\cdot 1.5 + 0.5\cdot 5) = \frac{1}{2}(-0.75+2.5) = 0.875$$

Using the learning rate $\alpha = 0.1$:

$$w_0 = 0 - 0.1 \cdot 0 = 0$$
$$w_1 = 0 - 0.1 \cdot 0.25 = -0.025$$
$$w_2 = 0 - 0.1 \cdot 0.875 = -0.0875$$

After the first iteration, the weights are:

$$w = (w_0, w_1, w_2) = (0, -0.025, -0.0875)$$

Step 2:

For $x_1 = (5, 1.5)$:

$$z_1 = w_0 + w_1 \cdot 5 + w_2 \cdot 1.5 = 0 + (-0.025)\cdot 5 + (-0.0875)\cdot 1.5$$
$$= -0.125 - 0.13125 = -0.25625$$

$$\hat{y}_1 = \sigma(z_1) = \frac{1}{1+e^{0.25625}} \approx 0.4363$$

For $x_2 = (6,5)$:

$$\begin{aligned} z_2 &= w_0 + w_1 \cdot 6 + w_2 \cdot 5 = 0 + (-0.025) \cdot 6 + (-0.0875) \cdot 5 \\ &= -0.15 - 0.4375 = -0.5875 \\ \hat{y}_2 &= \sigma(z_2) = \frac{1}{1+e^{0.5875}} \approx 0.3572 \end{aligned}$$

Gradient with respect to w_0:

$$\begin{aligned} \frac{\partial L}{\partial w_0} &= \frac{1}{2}((0.4363-1) + (0.3572-0)) \\ &= \frac{1}{2}(-0.5638 + 0.3572) = -0.10325 \end{aligned}$$

Gradient with respect to w_1:

$$\frac{\partial L}{\partial w_1} = \frac{1}{2}((0.4363-1)\cdot 5 + (0.3572-0)\cdot 6) = -0.33765$$

Gradient with respect to w_2:

$$\frac{\partial L}{\partial w_2} = \frac{1}{2}((0.4363-1)\cdot 1.5 + (0.3572-0)\cdot 5) = 0.470225$$

$$\begin{aligned} w_0 &= 0 - 0.1 \cdot (-0.10325) = 0.01025 \\ w_1 &= -0.025 - 0.1 \cdot -0.33765 = 0.008765 \\ w_2 &= -0.0875 - 0.1 \cdot 0.470225 = -0.1345225 \end{aligned}$$

After the second iteration, the weights are approximately:

$$w = (w_0, w_1, w_2) \approx (0.01025, 0.008765, -0.1345225)$$

ii) Suppose that a neural network has been trained using the training data found in Table 4, following the architecture and weights as illustrated in Figure 2.1. The trained network is structured with an input layer comprising two inputs, represented as (x_1) and (x_2), and bias. The hidden layer consists of bias and three neurons denoted as (a_1), (a_2), and (a_3), all using tanh activation functions. The output layer features a single neuron marked as (b), which utilises a sigmoid activation function. Apply feedforward operation to get the predicted values for the

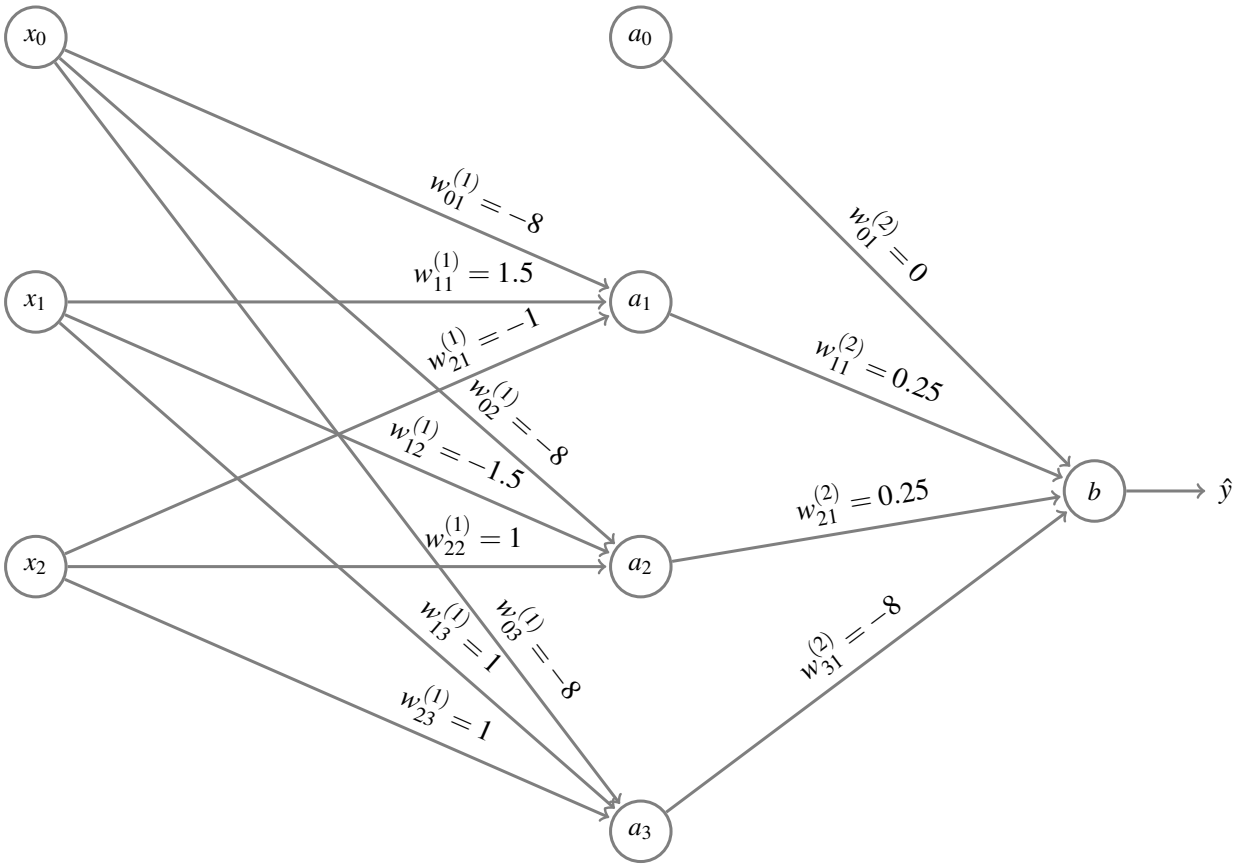


Figure 2.1 Trained neural network (x_0 *and* a_0 *are both equal to 1 and they are used to accomdate the bias term*).

training data, then estimate the training error using binary cross entropy error (BCE), and mention if overfitting is likely to occurr based on your results and the training scenario. **[20 marks]**

Solution: The feedforward operation can be represented using matrix notation as follows:

Feedforward Pass for $x_1 = (5, 1.5)$, first we calculate the signal for the hidden layer:

$$\begin{aligned}
\begin{bmatrix} S_{a_1} \\ S_{a_2} \\ S_{a_3} \end{bmatrix} &= \begin{bmatrix} w_{01}^{(1)} & w_{11}^{(1)} & w_{21}^{(1)} \\ w_{02}^{(1)} & w_{12}^{(1)} & w_{22}^{(1)} \\ w_{03}^{(1)} & w_{13}^{(1)} & w_{23}^{(1)} \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} \\
&= \begin{bmatrix} -8 & 1.5 & -1 \\ -8 & -1.5 & 1 \\ -8 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 5 \\ 1.5 \end{bmatrix} \\
&= \begin{bmatrix} -2 \\ -14 \\ -1.5 \end{bmatrix}
\end{aligned}$$

Calculating the activations on the hidden layer using $\theta(S) = \tanh(S)$ as follows:

$$\begin{aligned}
\begin{bmatrix} \theta_{a_1} \\ \theta_{a_2} \\ \theta_{a_3} \end{bmatrix} &= \begin{bmatrix} \tanh(S_{a_1}) \\ \tanh(S_{a_2}) \\ \tanh(S_{a_3}) \end{bmatrix} \\
&= \begin{bmatrix} \tanh(-2) \\ \tanh(-14) \\ \tanh(-1.5) \end{bmatrix} \\
&= \begin{bmatrix} -0.964 \\ -1 \\ -0.905 \end{bmatrix}
\end{aligned}$$

Calculating the signal for the output layer

$$S_b = \begin{bmatrix} w_{01}^{(2)} & w_{11}^{(2)} & w_{21}^{(2)} & w_{31}^{(2)} \end{bmatrix} \cdot \begin{bmatrix} a_0 \\ \theta_{a_1} \\ \theta_{a_2} \\ \theta_{a_3} \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0.25 & 0.25 & -8 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -0.964 \\ -1 \\ -0.905 \end{bmatrix}$$

$$= 6.749$$

Calculating the activation on the output layer using $\theta(S) = \sigma(S) = \frac{1}{1+e^{-S}}$ as follows:

$$\begin{aligned} \hat{y}_1 &= \theta_b \\ &= \frac{1}{1+e^{-S_b}} \\ &= \frac{1}{1+e^{-6.749}} \\ &= 0.999 \end{aligned}$$

Feedforward Pass for $x_2 = (6,5)$, first we calculate the signal for the hidden layer:

$$\begin{aligned} \begin{bmatrix} S_{a_1} \\ S_{a_2} \\ S_{a_3} \end{bmatrix} &= \begin{bmatrix} w_{01}^{(1)} & w_{11}^{(1)} & w_{21}^{(1)} \\ w_{02}^{(1)} & w_{12}^{(1)} & w_{22}^{(1)} \\ w_{03}^{(1)} & w_{13}^{(1)} & w_{23}^{(1)} \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} \\ &= \begin{bmatrix} -8 & 1.5 & -1 \\ -8 & -1.5 & 1 \\ -8 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 6 \\ 5 \end{bmatrix} \\ &= \begin{bmatrix} -4 \\ -12 \\ 3 \end{bmatrix} \end{aligned}$$

Calculating the activations on the hidden layer using $\theta(S) = \tanh(S)$ as follows:

$$\begin{aligned} \begin{bmatrix} \theta_{a_1} \\ \theta_{a_2} \\ \theta_{a_3} \end{bmatrix} &= \begin{bmatrix} \tanh(S_{a_1}) \\ \tanh(S_{a_2}) \\ \tanh(S_{a_3}) \end{bmatrix} \\ &= \begin{bmatrix} \tanh(-4) \\ \tanh(-12) \\ \tanh(3) \end{bmatrix} \end{aligned}$$

$$= \begin{bmatrix} -0.9993 \\ -1 \\ 0.9951 \end{bmatrix}$$

Calculating the signal for the output layer

$$\begin{aligned} S_b &= \begin{bmatrix} w_{01}^{(2)} & w_{11}^{(2)} & w_{21}^{(2)} & w_{31}^{(2)} \end{bmatrix} \cdot \begin{bmatrix} a_0 \\ \theta_{a_1} \\ \theta_{a_2} \\ \theta_{a_3} \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0.25 & 0.25 & -8 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -0.993 \\ -1 \\ 0.9951 \end{bmatrix} \\ &= -8.45905 \end{aligned}$$

Calculating the activation on the output layer using $\theta(S) = \sigma(S) = \frac{1}{1+e^{-S}}$ as follows:

$$\begin{aligned} \hat{y}_1 &= \theta_b \\ &= \frac{1}{1+e^{-S_b}} \\ &= \frac{1}{1+e^{8.45905}} \\ &= 0.000212 \end{aligned}$$

Calculating the training error using BCE:

$$\begin{aligned} \text{BCE} &= \frac{1}{N}\sum_{i=1}^{N}[-y_i \log(\hat{y}_i) - (1-y_i)\log(1-\hat{y}_i)] \\ &= \frac{1}{2}([-\log(0.999)] + [-\log(1-0.000212)]) \\ &= \frac{1}{2}(0.001 + 0.000212) \\ &= 0.000606 \\ &\approx 0 \end{aligned}$$

Overfitting is likely to occur because we get an almost zero training error on training using two data points only. The complexity of the model is not compatible with amount of data.

iii) Apply backpropagation algorithm with stochastic gradient descent to calculate one update of weight $w_{23}^{(1)}$ of the network in Figure 2.1 using the data point $x = [x_1, x_2] = [6, 5]$ with its label $y = 0$. Assume that we are using the same weights in the figure with the same activation functions as in the previous part and a learning rate of 0.1. For your analysis, consider using BCE. Show in your solution all the equations and steps (include all the δ's used in the back-propagation formula and update rule) and discuss the impact of the update and whether the difference is significant (with reasons why it is significant or not). **[15 marks]**

Solution:

Estimate δ_b:

$$\begin{aligned}
\delta_b &= \frac{\partial BCE}{\partial \hat{y}} \cdot \frac{\hat{y}}{\partial S_b} \\
&= \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \right) \cdot (\hat{y}(1-\hat{y})) \\
&= \left(-\frac{0}{0.000212} + \frac{1-0}{1-0.000212} \right) \cdot (0.000212(1-0.000212)) \\
&= 0.000212
\end{aligned}$$

Estimate δ_{a_3}:

$$\begin{aligned}
\delta_{a_3} &= \delta_b \cdot \frac{\partial S_b}{\partial \theta_{a_3}} \cdot \frac{\partial \theta_{a_3}}{\partial S_{a_3}} \\
&= \delta_b \cdot w_{31}^{(2)} \cdot (1-\theta_{a_3})^2 \\
&= 0.000212 \cdot -8 \cdot (1-0.9951)^2 \\
&= -4.0721e-8
\end{aligned}$$

Then we apply the update rule as follows:

$$\begin{aligned}
w_{23}^{(1)} &\leftarrow w_{23}^{(1)} - \eta \cdot \delta_{a_3} \cdot x_2 \\
w_{23}^{(1)} &\leftarrow 1 - (0.1 \cdot -4.0721e-8 \cdot 5) \\
w_{23}^{(1)} &\approx 1
\end{aligned}$$

There is no significant change in weight (almost negligible), that is because the error was very small.

b) Consider the data samples along with their ground truth values y shown in Table 5.

x	y
1	2
3	5
5	3

Table 5 Data sample.

i) Apply ridge regression with $\lambda = 0.1$ to estimate $\mathbf{W}_{reg}$. **[10 marks]**

Solution:

$$\mathbf{W}_{reg} = \left(\mathbf{X}^\top \mathbf{X} + \lambda I\right)^{-1} \mathbf{X}^\top \mathbf{y}$$

$$= \left(\begin{bmatrix} 1 & 1 & 1 \\ 1 & 3 & 5 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 3 \\ 1 & 5 \end{bmatrix} + 0.1 \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right)^{-1} \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 3 & 5 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 5 \\ 3 \end{bmatrix}$$

$$= \begin{bmatrix} 3.1 & 9 \\ 9 & 35.1 \end{bmatrix}^{-1} \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 3 & 5 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 5 \\ 3 \end{bmatrix}$$

$$= \begin{bmatrix} 2.2654 \\ 0.331 \end{bmatrix} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

ii) Using linear regression with leave-one-out cross validation. Estimate the total cross validation error such that each time the model is trained on $n-1$ samples will generate a training error of zero (use MSE function as your error function). **[20 marks]**

Solution:

The model will generate zero training error each time when trained on $n-1$ samples. That means the linear regressor line lies exactly on the two points when discarding the third on each fold:

Leaving first sample for validation $(x,y) = (1,2)$:

To generate zero training error in this case means the line for the regressor will lie exactly on the points $(3,5)$ and $(5,3)$, which give the following hypothesis $h_1 \rightarrow \mathbf{w} = [w_0, w_1] = [8,-1]$, we then can calculate the validation error on this fold as $\hat{R}_{v_1} = (2-(8+(-1\cdot 1))^2 = 25$

Leaving second sample for validation $(x,y) = (3,5)$:

To generate zero training error in this case means the line for the regressor will lie exactly on the points $(1,2)$ and $(5,3)$, which give the following hypothesis $h_2 \rightarrow \mathbf{w} = [w_0, w_1] = [1.75, 0.25]$, we then can calculate the validation error on this fold as $\hat{R}_{v_2} = (5-(1.75+(0.25\cdot 3))^2 = 6.25$

Leaving third sample for validation $(x,y) = (5,3)$:

To generate zero training error in this case means the line for the regressor will lie exactly on the points $(1,2)$ and $(3,5)$, which give the following hypothesis $h_3 \rightarrow \mathbf{w} = [w_0, w_1] = [0.5, 1.5]$, we then can calculate the validation error on this fold as $\hat{R}_{v_3} = (3-(0.5+(1.5\cdot 5))^2 = 25$

Total cross validation error is then

$$\begin{aligned}\hat{R}_{CV} &= \frac{1}{3}\left(\hat{R}_{v_1} + \hat{R}_{v_2} + \hat{R}_{v_3}\right) \\ &= \frac{1}{3}(25 + 6.25 + 25) \\ &= 18.75\end{aligned}$$

c) Assume that two neural networks were trained on the same dataset, the performance of both networks were evaluated based on the validation and training error values as shown in Figures 2.2 and 2.3.

i) Which model would you choose and why? **[5 marks]**

Solution: Model 2 is more feasible since it can generalize better - the complexity of the model seems to accommodate the size of the data - the validation error have the potential to improve

ii) Mention the feasibility of applying early stopping for the training in both networks, mention when and why? **[5 marks]**

Solution: This is feasible in Model 1, we can apply early stopping at epoch 200. However, Model 2 has the potential to improve so we should continue the training.

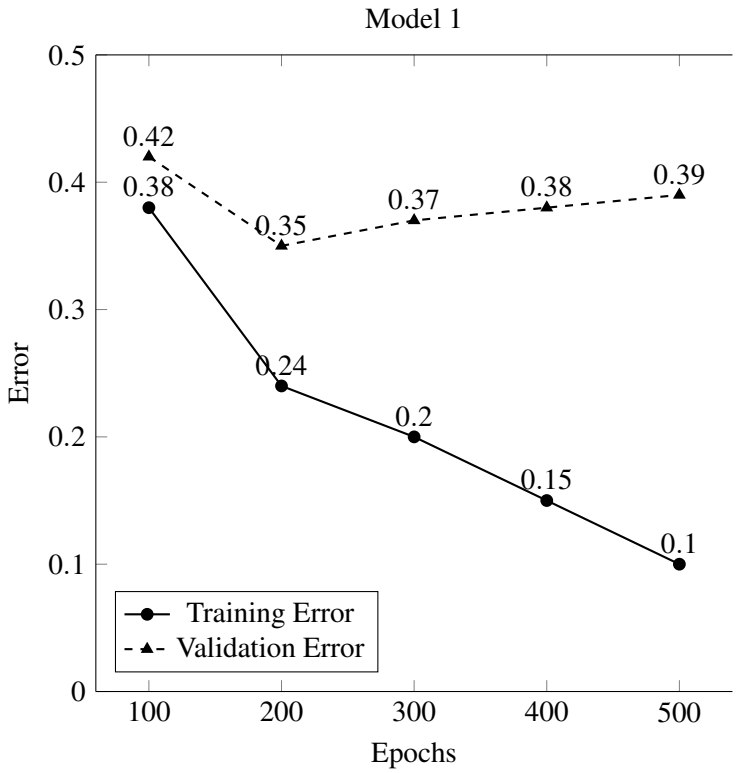


Figure 2.2 Neural network training on model 1.

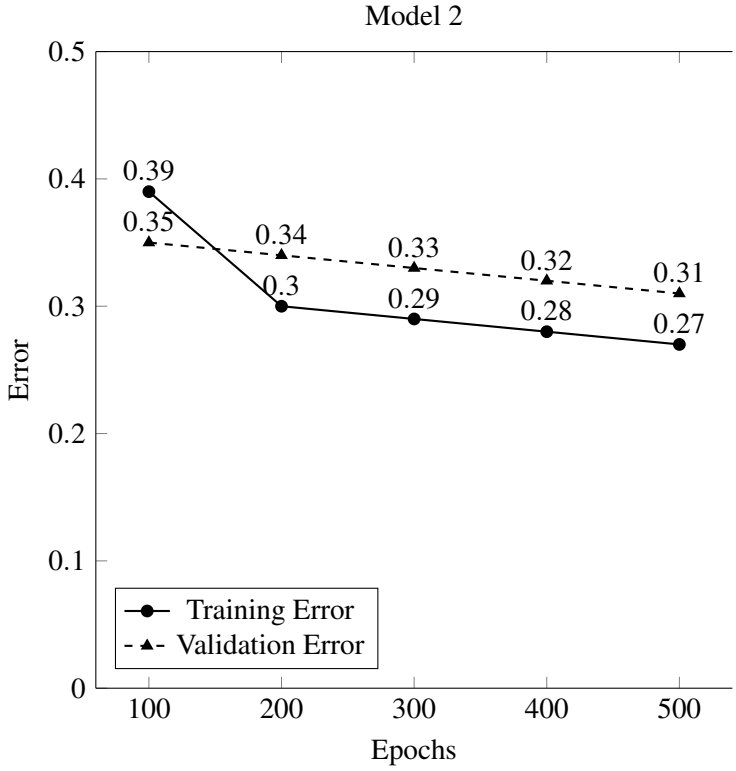


Figure 2.3 Neural network training on model 2.

iii) Which model is more likely that uses regularization? **[5 marks]**

Solution: Since both models are using the same data, we can tell that Model 2 is more likely that uses regularization because the overfitting is reduced when compared to Model 1.

3. Support vector machines, kernel methods

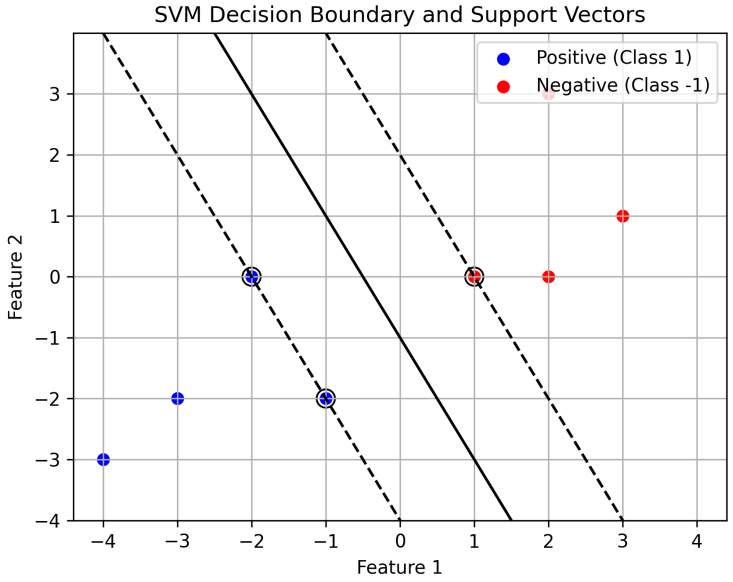
a) Consider the following dataset:

Positive examples: [-4, -3], [-3, -2], [-1, -2], [-2, 0]

Negative examples: [1, 0], [2, 3], [3, 1], [2, 0]

i) Mark these points on a plot, and identify the maximum margin classifier. Write down the equation of the corresponding hyperplane in the form $w^T x + b = 0$, where $\|w\|^2 = 1$. **[20 marks]**

Solution:



The hyperplane is characterized by $2x_1 + x_2 + 1 = 0$. To write in the desired form, we normalize by $\sqrt{5}$, and obtain

$$w = \begin{bmatrix} 2/\sqrt{5} \\ 1/\sqrt{5} \end{bmatrix}, \qquad b = 1/\sqrt{5}$$

ii) Identify the support vectors. **[5 marks]**

Solution: The three support vectors are $[-2,0], [-1,-2]$ and $[1,0]$.

b) Answer the following questions about support vector machines (SVMs).

i) Consider we have a linearly separable dataset of N samples in a d-dimensional space, and we apply hard-margin SVM. What are the

minimum and maximum number of support vectors? **[10 marks]**

Solution: The minimum number of support vectors is $d+1$, because $d+1$ points are required to define a unique hyperplane in a d-dimensional space. For example, in 2D, 3 points are needed to define the optimal margin, while in 3D, 4 points would be needed.

The maximum number of support vectors can reach up to N; that is, in the worst-case scenario, every data point could become a support vector.

ii) Suppose we use the soft-margin SVM formulation, and gradually increase the penalty parameter C. Do you expect the number of support vectors to increase or decrease as C increases? **[10 marks]**

Solution: In a soft-margin SVM, as we gradually increase the penalty parameter C, we expect the number of support vectors to decrease. The parameter C controls the trade-off between maximizing the margin and minimizing classification errors (or margin violations). When C is large, the SVM places a high penalty on misclassifications or violations of the margin. With a high C, the SVM will adjust the decision boundary to reduce the number of misclassified points, often at the expense of narrowing the margin. This results in fewer points lying within or on the margin boundaries, effectively reducing the number of support vectors.

iii) Suppose that we use the formulation for the soft-margin SVM, i.e., with slack variables ξ_i, but the data we have is actually linearly separable. Do we always recover the 'true' max-margin separating hyperplane? **[10 marks]**

Solution: This depends on the value of C, the coefficient we use to penalize the slack variables ξ_i's. If C is sufficiently large, we recover a hard margin classifier, which will give us the true max-margin separating hyperplane. However, if C is small, we might end up getting a different hyperplane with a larger margin, but with some data points on the wrong side of the hyperplane.

c) Let $\mathrm{K}(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ denote the kernel function for some feature map $\phi(\cdot)$ that maps data points to a higher dimensional feature space. Consider a

dataset of 3 points $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$, where the kernel matrix is given by:

$$\mathrm{K} = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

For example, $\mathrm{K}(\mathbf{x}_1, \mathbf{x}_1) = \phi(\mathbf{x}_1)^T \phi(\mathbf{x}_1) = 4$ and $\mathrm{K}(\mathbf{x}_1, \mathbf{x}_2) = \phi(\mathbf{x}_1)^T \phi(\mathbf{x}_2) = 2$, and so on so forth.

i) Prove that the Mercer conditions are satisfied for K for these three points. **[10 marks]**

Solution: Mercer condition requires K to be symmetric and positive semi-definite. K is symmetric. To show that it is positive semi-definite, we need to prove that $\mathbf{u}^T \mathrm{K} \mathbf{u} \geq 0$ for any vector $\mathbf{u}$. We can write

$$\begin{aligned}
\mathbf{u}^T \mathrm{K} \mathbf{u} &= [u_1 \;\; u_2 \;\; u_3] \begin{bmatrix} 4 & 2 & 1 \\ 2 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} [u_1 \;\; u_2 \;\; u_3]^T \\
&= [4u_1 + 2u_2 + u_3 \quad 2u_1 + 2u_2 + u_3 \quad u_1 + u_2 + u_3][u_1 \;\; u_2 \;\; u_3]^T \\
&= 4u_1^2 + 4u_1u_2 + 2u_1u_3 + 2u_2u_1 + 2 * u_2^2 + 2u_2u_3 + u_3u_1 + u_3u_2 + u_3^2 \\
&= (2u_1 + u_2 + u_3)^2 \\
&\geq 0.
\end{aligned}$$

ii) Find the squared norm of the mean vector in the feature space $\|\boldsymbol{\mu}_\phi\|^2$, where the mean vector is defined as:

$$\boldsymbol{\mu}_\phi = \frac{1}{3} \sum_{i=1}^{3} \phi(\mathbf{x}_i).$$

[15 marks]

Solution:

$$\begin{aligned}
\|\boldsymbol{\mu}_\phi\|^2 &= \boldsymbol{\mu}_\phi^T \boldsymbol{\mu}_\phi \\
&= \left(\frac{1}{3} \sum_{i=1}^{3} \phi(\mathbf{x}_i) \right)^T \left(\frac{1}{3} \sum_{j=1}^{3} \phi(\mathbf{x}_j) \right) \\
&= \frac{1}{9} \sum_{i=1}^{n} \sum_{j=1}^{3} \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)
\end{aligned}$$

$$= \frac{1}{9}\sum_{i=1}^{3}\sum_{j=1}^{3} \mathrm{K}(\mathbf{x}_i, \mathbf{x}_j)$$

which is the average of the values of the Kernel matrix K, given by $15/9 = 1.67$.

iii) Find the total variance in the feature space, defined as:

$$\sigma_\phi^2 = \frac{1}{3}\sum_{i=1}^{3} \|\phi(\mathbf{x}_i) - \boldsymbol{\mu}_\phi\|^2$$

[20 marks]

Solution: We have

$$\begin{aligned}\|\phi(\mathbf{x}_i) - \boldsymbol{\mu}_\phi\|^2 &= \|\phi(\mathbf{x}_i)\|^2 - 2\phi(\mathbf{x}_i)^T\boldsymbol{\mu}_\phi + \|\boldsymbol{\mu}_\phi\|^2 \\ &= \mathrm{K}(\mathbf{x}_i, \mathbf{x}_i) - \frac{2}{n}\sum_{j=1}^{n} K(\mathbf{x}_i, \mathbf{x}_j) + \frac{1}{n^2}\sum_{a=1}^{n}\sum_{b=1}^{n} K(\mathbf{x}_a, \mathbf{x}_b)\end{aligned}$$

Then, by taking the average value, we get

$$\begin{aligned}\sigma_\phi^2 &= \frac{1}{3}\sum_{i=1}^{3} \|\phi(\mathbf{x}_i) - \boldsymbol{\mu}_\phi\|^2 \\ &= \frac{1}{3}\sum_{i=1}^{3}\left(\mathrm{K}(\mathbf{x}_i, \mathbf{x}_i) - \frac{2}{3}\sum_{j=1}^{3}\mathrm{K}(\mathbf{x}_i, \mathbf{x}_j) + \frac{1}{9}\sum_{a=1}^{3}\sum_{b=1}^{3} K(\mathbf{x}_a, \mathbf{x}_b)\right) \\ &= \frac{1}{3}\sum_{i=1}^{3} K(\mathbf{x}_i, \mathbf{x}_i) - \frac{2}{9}\sum_{i=1}^{3}\sum_{j=1}^{3}\mathrm{K}(\mathbf{x}_i, \mathbf{x}_j) + \frac{1}{9}\sum_{a=1}^{3}\sum_{b=1}^{3} K(\mathbf{x}_a, \mathbf{x}_b) \\ &= \frac{1}{3}\sum_{i=1}^{3}\mathrm{K}(\mathbf{x}_i, \mathbf{x}_i) - \frac{1}{9}\sum_{i=1}^{3}\sum_{j=1}^{3}\mathrm{K}(\mathbf{x}_i, \mathbf{x}_j)\end{aligned}$$

Hence, we obtain $\sigma_\phi^2 = 7/3 + 5/3 = 4$.

4. Principal components analysis, clustering.

a) You are given the following data set:

$$\mathbf{x_1} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \mathbf{x_2} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad \mathbf{x_3} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

i) Plot the data set. **[3 marks]**

ii) Compute the empirical covariance matrix **S**. **[6 marks]**

Solution:

$$\mathbf{S} = \frac{\mathbf{1}}{\mathbf{3}}\mathbf{X^T X} = \frac{\mathbf{1}}{\mathbf{3}} \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ -1 & 1 \end{pmatrix}^{\mathbf{T}} \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ -1 & 1 \end{pmatrix} = \begin{pmatrix} 6 & 1 \\ 1 & 2 \end{pmatrix}$$

iii) Compute the eigenvectors and corresponding eigenvalues S. What is the unnormalized basis vector $\mathbf{u}_1$ of your 1-dimensional representation? **[10 marks]**

Solution: The eigenvectors of the matrix $\begin{bmatrix} 6 & 1 \\ 1 & 2 \end{bmatrix}$ are:

$$\begin{bmatrix} 0.9732 & -0.2298 \\ 0.2298 & 0.9732 \end{bmatrix}$$

Each column represents an eigenvector of the matrix. The corresponding eigenvalues are $\lambda_1 = 6.2361$ and $\lambda_2 = 1.7639$.

The unnormalized basis can be computed as follows:

For λ_1, we have The eigenvector v_1 satisfies the equation:

$$(S - \lambda_1 I)v_1 = 0$$

Substitute $\lambda_1 = 6.2361$ into the equation:

$$S - \lambda_1 I = \begin{bmatrix} 6 - 6.2361 & 1 \\ 1 & 2 - 6.2361 \end{bmatrix} = \begin{bmatrix} -0.2361 & 1 \\ 1 & -4.2361 \end{bmatrix}$$

The system of equations is:

$$\begin{bmatrix} -0.2361 & 1 \\ 1 & -4.2361 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This gives the equation:

$$-0.2361x + y = 0$$

Rearranging, we find:

$$y = 0.2361x$$

Thus, the unnormalized eigenvector corresponding to $\lambda_1 = 6.2361$ is:

$$v_1 = \begin{bmatrix} 1 \\ 0.2361 \end{bmatrix}$$

For $\lambda_2 = 1.7639$:

The eigenvector v_2 satisfies the equation:

$$(S - \lambda_2 I)v_2 = 0$$

Substitute $\lambda_2 = 1.7639$ into the equation:

$$S - \lambda_2 I = \begin{bmatrix} 6 - 1.7639 & 1 \\ 1 & 2 - 1.7639 \end{bmatrix} = \begin{bmatrix} 4.2361 & 1 \\ 1 & 0.2361 \end{bmatrix}$$

The system of equations is:

$$\begin{bmatrix} 4.2361 & 1 \\ 1 & 0.2361 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This gives the equation:

$$4.2361x + y = 0$$

Rearranging, we find:

$$y = -4.2361x$$

Thus, the unnormalized eigenvector corresponding to $\lambda_2 = 1.7639$ is:

$$v_2 = \begin{bmatrix} 1 \\ -4.2361 \end{bmatrix}$$

Summarily, for $\lambda_1 = 6.2361$: $v_1 = \begin{bmatrix} 1 \\ 0.2361 \end{bmatrix}$ and for $\lambda_2 = 1.7639$: $v_2 = \begin{bmatrix} 1 \\ -4.2361 \end{bmatrix}$

iv) Compute the coefficients z_1, z_2, z_3 of the three samples in the dataset. Compute the lower-dimensional representations $z_1\mathbf{v_1}$, $z_2\mathbf{v_1}$, $z_3\mathbf{v_1}$. **[10 marks]**

Solution: We compute the coefficients z_1, z_2, z_3 of the projections of the data points onto the eigenvectors $\mathbf{v_1}$ and $\mathbf{v_2}$.

Projection of $\mathbf{x_1}$

$$\mathbf{x_1}^T\mathbf{v_1} = 1 \times 0.9732 + 0 \times 0.2298 = 0.9732$$

$$\mathbf{x_1}^T\mathbf{v_2} = 1 \times 0.2298 + 0 \times (-0.9732) = 0.2298$$

Thus, the coefficients for $\mathbf{x_1}$ are:

$$z_{11} = 0.9732, \quad z_{12} = 0.2298$$

Projection of $\mathbf{x_2}$

$$\mathbf{x_2}^T\mathbf{v_1} = 2 \times 0.9732 + 1 \times 0.2298 = 1.9464 + 0.2298 = 2.1762$$

$$\mathbf{x_2}^T\mathbf{v_2} = 2 \times 0.2298 + 1 \times (-0.9732) = 0.4596 - 0.9732 = -0.5136$$

Thus, the coefficients for $\mathbf{x_2}$ are:

$$z_{21} = 2.1762, \quad z_{22} = -0.5136$$

Projection of $\mathbf{x_3}$

$$\mathbf{x_3}^T\mathbf{v_1} = -1 \times 0.9732 + 1 \times 0.2298 = -0.9732 + 0.2298 = -0.7434$$

$$\mathbf{x_3}^T\mathbf{v_2} = -1 \times 0.2298 + 1 \times (-0.9732) = -0.2298 - 0.9732 = -1.2030$$

Thus, the coefficients for $\mathbf{x_3}$ are:

$$z_{31} = -0.7434, \quad z_{32} = -1.2030$$

Lower-Dimensional Representations Using the coefficients z_{i1} and z_{i2}, we compute the lower-dimensional representations of the data points by projecting them onto the eigenvectors.

For $\mathbf{x_1}$:

$$z_1\mathbf{v_1} = 0.9732 \times \begin{bmatrix} 0.9732 \\ 0.2298 \end{bmatrix} = \begin{bmatrix} 0.9478 \\ 0.2237 \end{bmatrix}$$

$$z_1\mathbf{v_2} = 0.2298 \times \begin{bmatrix} 0.2298 \\ -0.9732 \end{bmatrix} = \begin{bmatrix} 0.0528 \\ -0.2237 \end{bmatrix}$$

For $\mathbf{x_2}$:

$$z_2\mathbf{v_1} = 2.1762 \times \begin{bmatrix} 0.9732 \\ 0.2298 \end{bmatrix} = \begin{bmatrix} 2.1206 \\ 0.5008 \end{bmatrix}$$

$$z_2\mathbf{v_2} = -0.5136 \times \begin{bmatrix} 0.2298 \\ -0.9732 \end{bmatrix} = \begin{bmatrix} -0.1187 \\ 0.5007 \end{bmatrix}$$

For $\mathbf{x_3}$:

$$z_3\mathbf{v_1} = -0.7434 \times \begin{bmatrix} 0.9732 \\ 0.2298 \end{bmatrix} = \begin{bmatrix} -0.7233 \\ -0.1708 \end{bmatrix}$$

$$z_3\mathbf{v_2} = -1.2030 \times \begin{bmatrix} 0.2298 \\ -0.9732 \end{bmatrix} = \begin{bmatrix} -0.2762 \\ 1.1703 \end{bmatrix}$$

v) Why would we want to reduce the dimensionality of our data? Can you think of examples where this can be useful? **[5 marks]**

Solution: There can be several reasons: i) Fewer features are easier to interpret: we might want to know why our model outputs a certain diagnosis, and only some of the patient record details will be relevant. ii) Models with fewer features are easier to handle computationally. iii) Our data might be arbitrarily high-dimensional because of noise, so we would like to access the lower-dimensional information-carrying signal from the data.

vi) Consider the following design matrix, representing four sample points $X_i \in \mathbb{R}^2$. $X = \{(4,1),(2,3),(5,4),(1,0)\}$. We want to represent the data in only one dimension, so we turn to principal components analysis (PCA). Compute the unit-length principal component directions of X, and state which one the PCA algorithm would choose if you request just one principal component. Please provide an exact answer, without approximation. (You will need to use the square root symbol.) Show your work! **[3 marks]**

Solution: We center X, yielding $X = \{(1,-1),(-1,1),(2,2),(-2,-2)\}$. Then $X^TX = [[10,6],[6,10]]$. (Can also divide by 4 if you want the sample covariance but we do not care about the magnitude here). Its eigenvectors are $[1/\sqrt{2} \quad 1/\sqrt{2}]^T$ with eigenvalue 16 and $[1/\sqrt{2} \quad -1/\sqrt{2}]^T$ with eigenvalue 4. The former eigenvector is chosen.

vii) The plot in Figure 4.1 depicts the sample points from X. We want a one-dimensional representation of the data, so draw the principal component direction (as a line) and the projections of all four sample points onto the principal direction. Label each projected point with its principal coordinate value (where the origin's principal coordinate is zero). Give the principal coordinate values exactly. **[3 marks]**

Note: Transfer the plot shown in Figure 4.1 into your answer book.

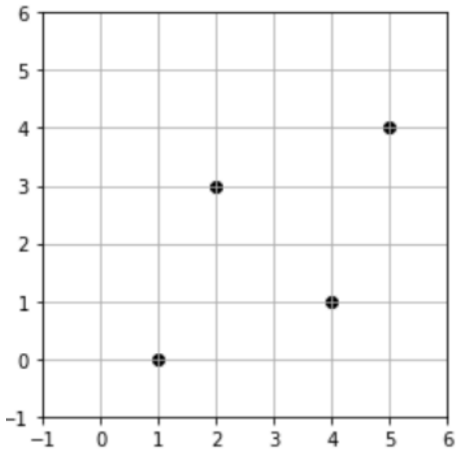
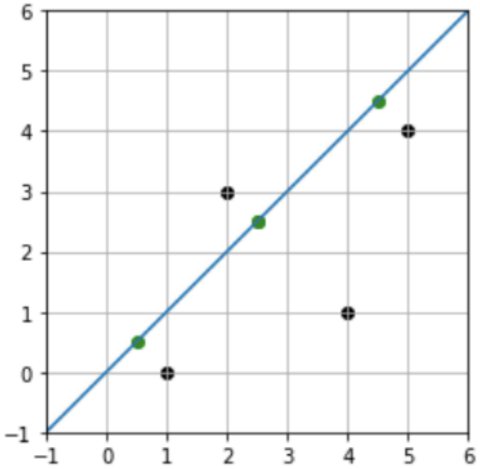


Figure 4.1 Sample points from X.

Solution:



The principal coordinates are $1/\sqrt{2}, 5/\sqrt{2}, 5/\sqrt{2}, 9/\sqrt{2}$. (Alternatively, all of these could be negative, but they all have to have the same sign.)

viii) Summarize how you would perform PCA on a dataset $X = \{\mathbf{x_i}\}_{\mathbf{i=1}}^{\mathbf{n}}$

and work with transformed dataset $\{\mathbf{z_i}\}_{\mathbf{i=1}}^{\mathbf{n}}$. **[15 marks]**

Solution:

- Calculate the empirical covariance matrix S after transforming the data to zero-mean,
- Decide how many dimensions K out of the original D we want to keep in the final representation,
- Find the K largest eigenvalues of S.
- The K eigenvectors $(\mathbf{u_1}, \ldots, \mathbf{u_K})$ corresponding to these eigenvalues will be our lower-dimensional basis. The corresponding basis transformation/projection matrix is thus $\mathbf{U} = [\mathbf{u_1}, \ldots, \mathbf{u_K}]$,
- Reduce the dimensionality of a data point $\mathbf{x}$ by projecting it onto the new basis, yielding a new reconstructed vector $\mathbf{z}$: $\mathbf{z} = \mathbf{U^T x}$

b) i) Consider a dataset $\mathscr{D} = \{x_1, x_2, \ldots, x_n\}$ in $\mathbb{R}^d$, and you are tasked with performing hierarchical clustering. Define hierarchical clustering and describe the difference between agglomerative and divisive hierarchical clustering approaches. **[5 marks]**

Solution:

Hierarchical clustering is a method of cluster analysis which seeks to build a hierarchy of clusters. It is often visualized as a dendrogram, where each level represents a merge (in agglomerative clustering) or a split (in divisive clustering).

- **Agglomerative Clustering**: This is a "bottom-up" approach. Each observation starts in its own cluster, and pairs of clusters are merged as one moves up the hierarchy based on their similarity.
- **Divisive Clustering**: This is a "top-down" approach. All observations start in one cluster, and the cluster is recursively split into smaller clusters based on dissimilarities.

ii) Explain the concept of linkage criteria in hierarchical clustering. List and briefly describe at least three different linkage methods. **[5 marks]**

Solution:

The linkage criterion in hierarchical clustering determines how the distance

between two clusters is computed, which in turn affects the clustering structure.

- **Single Linkage**: The distance between two clusters is defined as the minimum distance between any pair of points, with one point in each cluster.
- **Complete Linkage**: The distance between two clusters is defined as the maximum distance between any pair of points, with one point in each cluster.
- **Average Linkage**: The distance between two clusters is defined as the average distance between all pairs of points, where each pair consists of one point from each cluster.

iii) Given a distance matrix D for a dataset with four points, A, B, C, and D, use the single linkage method to perform agglomerative hierarchical clustering. The distance matrix D is:

$$D = \begin{pmatrix} 0 & 2 & 6 & 10 \\ 2 & 0 & 5 & 9 \\ 6 & 5 & 0 & 4 \\ 10 & 9 & 4 & 0 \end{pmatrix}$$

Present the clustering process as a dendrogram and indicate the order of merges. **[10 marks]**

Solution:

Using the given distance matrix D and the single linkage method:

i) Start with each point as its own cluster: $\{A\}$, $\{B\}$, $\{C\}$, $\{D\}$.

ii) **Step 1**: Find the closest pair of clusters. The smallest distance is $d(A,B) = 2$. Merge A and B into cluster $\{A,B\}$.

iii) **Step 2**: Update the distance matrix for the new clusters:

Distances to $\{A,B\}$:

$$d(\{A,B\},C) = \min(d(A,C),d(B,C)) = \min(6,5) = 5$$

$$d(\{A,B\},D) = \min(d(A,D),d(B,D)) = \min(10,9) = 9$$

Updated distance matrix:

$$\begin{pmatrix} 0 & 5 & 9 \\ 5 & 0 & 4 \\ 9 & 4 & 0 \end{pmatrix}$$

iv) **Step 3**: The next closest pair is $d(C,D) = 4$. Merge C and D into cluster $\{C,D\}$.

v) **Step 4**: Update the distance matrix:

Distances to $\{C,D\}$:

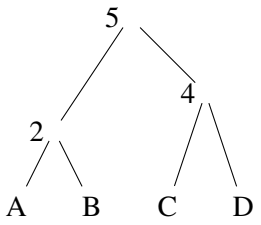
$d(\{A,B\},\{C,D\}) = \min(d(\{A,B\},C), d(\{A,B\},D)) = \min(5,9) = 5$

Updated distance matrix:

$$\begin{pmatrix} 0 & 5 \\ 5 & 0 \end{pmatrix}$$

vi) **Step 5**: Merge the last two clusters $\{A,B\}$ and $\{C,D\}$.

The dendrogram for this process can be illustrated as follows (with the vertical axis representing distance at each merge level):



Order of merges:

i) $\{A\}$ and $\{B\}$ at distance 2

ii) $\{C\}$ and $\{D\}$ at distance 4

iii) $\{A,B\}$ and $\{C,D\}$ at distance 5

com

iv) Discuss the strengths and weaknesses of hierarchical clustering compared to k-means clustering. In what scenarios would hierarchical clustering be preferred? **[5 marks]**

Solution: Strengths of Hierarchical Clustering:

- **No need to specify the number of clusters**: Unlike k-means, hierarchical clustering does not require the number of clusters k to be predefined.
- **Dendrogram visualization**: Allows examination of the data's structure at multiple levels, which is useful for exploratory analysis.
- **Flexible with various linkage criteria**: Allows experimenting with different linkage methods for different clustering behaviors.

Weaknesses:

- **Computational Complexity**: Agglomerative clustering can be computationally intensive (typically $O(n^2 \log n)$) and may not scale well for large datasets.
- **Sensitivity to noise and outliers**: Noisy points or outliers can distort the hierarchy since every point is involved in the clustering.

Comparison with k-means:

- **Hierarchical clustering** is preferred when we want to understand the nested structure of the data or when the number of clusters is unknown. It's also better suited for smaller datasets due to its higher complexity.
- **k-means** is preferable for large datasets where the number of clusters is known and clusters are roughly spherical in shape.

c) Suppose you are given the dataset in Table 6 with five points in a 2D space:

Point	x	y
A	1	1
B	1	4
C	5	1
D	5	4
E	9	8

Table 6 Sample points in a 2D space.

i) Use the K-means clustering algorithm with $k = 2$ to partition these points into two clusters. **[1 marks]**

(ii) Let the initial centroids for the two clusters be chosen as:

- Centroid 1: Point A (1, 1)
- Centroid 2: Point E (9, 8)

Assign each point to the nearest centroid. **[1 marks]**

iii) Perform **one iteration** of the K-means algorithm, showing each step clearly:

- Calculate the Euclidean distance of each point to each centroid.
- Assign each point to the nearest centroid.
- Compute the new centroids after this assignment.

[2 marks]

iv) Provide the final cluster assignments for this iteration and the new centroids.
[1 marks]

Solution:

i) **Calculate the Euclidean distance of each point to each centroid:**

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Distance from A (1, 1) to:
  - Centroid 1 (1, 1): $\sqrt{(1-1)^2 + (1-1)^2} = 0$
  - Centroid 2 (9, 8): $\sqrt{(9-1)^2 + (8-1)^2} = \sqrt{64+49} = \sqrt{113} \approx 10.63$
- Distance from B (1, 4) to:
  - Centroid 1 (1, 1): $\sqrt{(1-1)^2 + (4-1)^2} = \sqrt{9} = 3$
  - Centroid 2 (9, 8): $\sqrt{(9-1)^2 + (8-4)^2} = \sqrt{64+16} = \sqrt{80} \approx 8.94$
- Distance from C (5, 1) to:
  - Centroid 1 (1, 1): $\sqrt{(5-1)^2 + (1-1)^2} = \sqrt{16} = 4$
  - Centroid 2 (9, 8): $\sqrt{(9-5)^2 + (8-1)^2} = \sqrt{16+49} = \sqrt{65} \approx 8.06$
- Distance from D (5, 4) to:
  - Centroid 1 (1, 1): $\sqrt{(5-1)^2 + (4-1)^2} = \sqrt{16+9} = \sqrt{25} = 5$
  - Centroid 2 (9, 8): $\sqrt{(9-5)^2 + (8-4)^2} = \sqrt{16+16} = \sqrt{32} \approx 5.66$
- Distance from E (9, 8) to:
  - Centroid 1 (1, 1): $\sqrt{(9-1)^2 + (8-1)^2} = \sqrt{113} \approx 10.63$
  - Centroid 2 (9, 8): $\sqrt{(9-9)^2 + (8-8)^2} = 0$

ii) **Assign each point to the nearest centroid:**

- Point A (1, 1): Closest to Centroid 1
- Point B (1, 4): Closest to Centroid 1
- Point C (5, 1): Closest to Centroid 1
- Point D (5, 4): Closest to Centroid 2
- Point E (9, 8): Closest to Centroid 2

So, the clusters are:

- Cluster 1: Points A, B, and C
- Cluster 2: Points D and E

iii) **Compute the new centroids:**

- New Centroid 1 (for Cluster 1: A, B, C):

  $$\text{New x-coordinate} = \frac{1+1+5}{3} = 2.33, \quad \text{New y-coordinate} = \frac{1+4+1}{3} = 2$$

  So, Centroid 1 = (2.33, 2).

- New Centroid 2 (for Cluster 2: D, E):

  $$\text{New x-coordinate} = \frac{5+9}{2} = 7, \quad \text{New y-coordinate} = \frac{4+8}{2} = 6$$

  So, Centroid 2 = (7, 6).

iv) **Final Answer:**

- Cluster assignments for this iteration:
  - Cluster 1: Points A, B, and C
  - Cluster 2: Points D and E
- New centroids after one iteration:
  - Centroid 1: (2.33, 2)
  - Centroid 2: (7, 6)

c) Let us try and classify data points in 2D Euclidean space. We are given n instances of such points $p_1, p_2, \ldots, p_n$ and corresponding categories $c_1, c_2, \ldots, c_n$ where $c_1, c_2, \ldots, c_n$ take values from all possible class labels. Under the k-nearest neighbours classification scheme, each new element Q is simply categorized by a majority vote among its k nearest neighbors in instance space. The 1-NN is a simple variant of this which divides up the input space for classification purposes into a convex region (see Figure 4.2 for the 1NN decision boundaries under the Euclidean distance measure), each corresponding to a point in the instance set.

i) is it possible to build a decision tree (with decisions at each node of the form is "$x > a$", "$x < b$", "$y > c$", "$y < d$" for any real constants a, b, c, d which classifies exactly according to the 1-NN scheme using Euclidean distance? If so, explain how. If not, explain why not. **[5 marks]**

Solution: No. the decision boundaries for 1-NN correspond to the cell boundaries of each point (see image) and are not necessarily parallel to the coordinate axes. The decision tree boundaries would always be parallel to coordinate axes based on the kinds of questions asked at each node of the decision tree. To approximate a gradient by decision trees could take an arbitrary (uncountable) number of decisions, not possible in a decision tree.

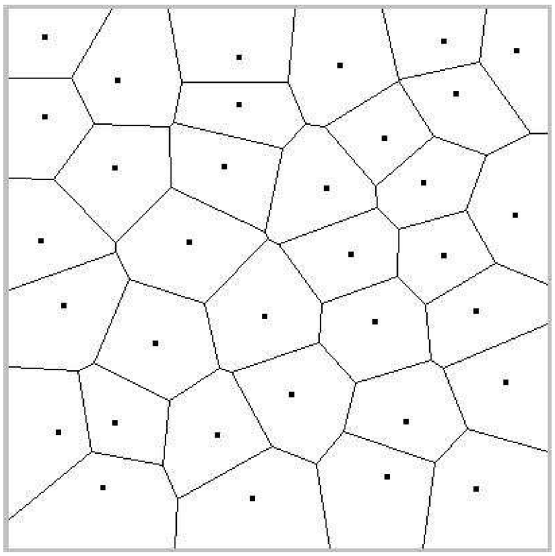


Figure 4.2 Decision boundaries.

ii) Now assume that the distance measure is not explicitly specified to you. Instead, you are given a "black box" where you input a set of instances $p_1, p_2, \ldots, p_n$ and a new example Q, and the black box outputs the nearest neighbor of Q, say π and its corresponding class label c_i. Is it possible to construct a k-NN classification algorithm based on this black box alone? If so, how and if not, why not? **[5 marks]**

Solution: Yes. First use the 1NN algorithm on the instance set for the new example, note the nearest neighbor and its class and discard it from the instance set. Use 1NN now with the reduced instance set and the example to be classified and again note the nearest neighbor and its class and throw it out of the instance set. Repeat this process k times, take a majority vote among the noted classes, and classify the new example accordingly.

iii) If the black box returns the j nearest neighbors (and their corresponding class labels) instead of the single most nearest neighbor (assume $j! = k$) , is it possible to construct a k-NN classification algorithm based on the black box ? If so how, and if not why not ? **[5 marks]**

Solution: If $j < k$, then use the algorithm $\lfloor k/j \rfloor$ times to obtain the $j * \lfloor k/j \rfloor$ nearest neighbors and their classes. To obtain the remaining $k–j * \lfloor k/j \rfloor$ nearest neighbors use the j-NN one more time and note the final batch of j nearest neighbors. Now to order the last set of j nearest neighbors and choose the top $k–j * \lfloor k/j \rfloor$ nearest neighbors from them. To do this, we merely need to

construct an instance set using the final batch of j nearest neighbors and any $j–(k–j*\lfloor k/j \rfloor)$ nearest neighbors from the set of the top j nearest neighbors. The $(k–j*\lfloor k/j \rfloor)$ elements from the last batch which get picked as the j nearest neighbors are thus the top $k–j*\lfloor k/j \rfloor$ elements in the last batch of j nearest neighbors that we needed to identify, where $\lfloor \cdot \rfloor$ denotes the floor function.

Note that if however $k < j$, we cannot use the j-NN algorithm black box.